

Section 1 / Q2 — Single Source Shortest Path (SSSP) with MapReduce

Finds the shortest distance from node 0 to every node in a weighted directed graph, using iterative MapReduce (parallel Bellman-Ford). Written in Python, orchestrated with Bash. Works both on a laptop and on a Slurm cluster.

Files

File	What it does
<code>prep.py</code>	Turns the raw edge list into the starting node state
<code>mapper.py</code>	One relaxation step: re-emits the graph + proposes new distances
<code>combiner.py</code>	Merges duplicate proposals locally before the shuffle (optimisation)
<code>reducer.py</code>	Picks the minimum distance per node, counts how many changed
<code>format_output.py</code>	Prints the final node distance lines, INF if unreachable
<code>sssp_local.sh</code>	Runs the whole thing on one machine
<code>sssp_slurm.sh</code>	Runs the whole thing on a Slurm cluster
<code>bench_slurm.sh</code>	Optional: times the pipeline over several graph sizes and task counts
<code>gen_graph.py</code>	Makes random test graphs (same seed = same graph)
<code>dijkstra_ref.py</code>	Ordinary sequential Dijkstra, used to check our answers
<code>verify.sh</code>	Runs all tests and compares against Dijkstra
<code>tests/</code>	Small hand-made graphs (sample from the PDF, chain, cycle, etc.)

PROF

Input / output format

Input file:

```
V E      <- number of vertices, number of edges
u v w    <- one line per edge: from u, to v, weight w
...
```

Output:

```
0 0
1 3
```

```
2 2
3 7          <- "node_id distance", sorted by node, INF if unreachable
```

Running locally (your own computer)

You only need **python3** and **bash**. No Hadoop, no cluster.

Step 1 — go to this folder

```
cd "Section 1"
```

Step 2 — run on the sample from the assignment PDF

```
./sssp_local.sh tests/sample.txt output.txt
cat output.txt
```

You should see:

```
0 0
1 3
2 2
3 7
```

Step 3 — run on a bigger graph

First create one (**gen_graph.py V E seed**), then run on it:

```
python3 gen_graph.py 10000 50000 7 > big.txt
./sssp_local.sh big.txt output.txt 4
```

The last number (**4**) is how many map tasks to simulate. Any number works; the answer is always the same. To use your own graph file, just put its name in place of **big.txt**.

Step 4 — check correctness against Dijkstra

```
./verify.sh
```

Prints **PASS/FAIL** for each test and ends with **ALL TESTS PASSED**.

Making a big graph for timing

```
python3 gen_graph.py 10000 50000 7 > big.txt      # V=10000, E=50000,
seed=7
./sssp_local.sh big.txt big_out.txt 4
python3 dijkstra_ref.py < big.txt > big_ref.txt
diff big_out.txt big_ref.txt && echo MATCH
```

If you get **Permission denied**, run `chmod +x *.sh *.py` once.

How it works (short version)

Each node's state is one line: `node \t dist|neighbour,weight;neighbour,weight;...`

Each iteration is one MapReduce job:

1. **Mapper** — for every node it re-emits the node's own line (so the graph survives to the next round) and, if the node is reachable, emits `neighbour \t D|dist+weight` for every out-edge. That is one Bellman-Ford relaxation.
2. **Shuffle (sort)** — groups everything for the same node together.
3. **Combiner** — on each mapper's output, collapses repeated proposals for the same node into one minimum, so less data crosses the network.
4. **Reducer** — for each node takes the minimum of its old distance and all proposals, writes the new line, and counts how many nodes improved.
5. The driver script repeats until the reducer reports **0 improvements** — at that point no edge can be relaxed further, so the distances are final.

Unreachable nodes never receive a proposal, stay at `1000000`, and are printed as `INF`. Converges in about the graph's hop-diameter rounds (22 rounds for $V=10000, E=50000$), never more than V .

—
PROF

Correctness

All twelve cluster configurations (three graph sizes x four task counts) came out correct, and for each graph the iteration count was identical at 1, 2, 4 and 8 map tasks (14, 22 and 25 respectively) -- the answer does not depend on how the input is split.

`verify.sh` compares the MapReduce output byte-for-byte with `dijkstra_ref.py` on: the assignment sample, a chain, a cycle, an edgeless graph, a graph with an unreachable component, and 3 random 150-node graphs — each with 1, 2 and 3 map tasks, confirming the result does not depend on how the input is split. A $V=10000 / E=50000$ random graph also matches exactly.